

The "Persona" Requirement

Role of the Persona

A high-quality rubric is objective, grounded in the task, and easy to apply consistently. To achieve this, your task requirements now include a **Persona**, which acts as the third pillar alongside your **Domain** and **Lens**.

We are looking for **creative, human-like, and original** trajectories that feel like a real person managing a complex life. A good rubric is a simple, reliable checklist to judge how well the model performed, and a successful persona-driven task makes those checks more meaningful.

While the Domain (e.g., Shopping) and Lens (e.g., Consumer) define the world, the Persona defines the **user's voice, information needs, and historical context**.

Accessing Persona Data

In your task details, you will see the **Name** of your assigned Persona. You must use the [Persona Visualizer](#) to retrieve the full profile. This tool is your Single Source of Truth (SSOT) for:

- **Identity Data:** Name, Email, and Address. While the agent is "logged in," you use these to verify that the tool calls (like `get_current_user`) are returning the correct persona information.
- **Lifestyle & Interests:** Biography and hobbies, use these to craft the narrative of why the user is seeking information (e.g., a "tech enthusiast" asking for the most advanced specs currently available).
- **Historical Data:** Past purchases and spending, essential for creating "Contextually Chained" turns that reference previous user behavior, such as when the user asks to compare new options against things they already own or to check the status of a previous order.

Handling the Information: Creativity vs. Data

Explicit Data (Static Parameters)

The agent already has access to the Persona information specified in the task details. **It is not necessary to add personal information (like email or phone) to the prompts.** Simply pretend you are that person in a logged-in state.

- **Active Session:** The active persona's details are fetched directly from the mock server. For example, calling `amazon_get-orders-history` will automatically display that specific persona's data.

Implicit Inspiration (Semantic Intent)

Use "Interests," "Personality Traits," and "Summary" fields to craft **original, human-like scenarios that justify a data-retrieval request.**

- **Instead of:** "Show me the characteristics of all available laptops."
- **Marcus Johnson Prompt:** "I'm looking to optimize my gaming setup for better frame rates. Can you retrieve the technical specs for the highest-rated GPUs currently in stock and compare them to the one I purchased last year?"

Specialty Stores & Multi-Server Strategy

You are not strictly limited to the specialty stores listed in the Persona's profile, but you must use them to ground the agent's search behavior.

- **Strategic Retrieval:** Keep the persona's "Specialty Stores" in mind when deciding where to retrieve information. For example, if Marcus Johnson is looking for tech specs, his first stop should logically be the `pc_parts` or `electronics_store` listed in his profile.
- **Mock vs. Live Servers:** You can navigate both Mock Data servers (for persona-specific history) and Live servers (for current market data) during the trajectory.
- **Contextual Awareness:** You must review the Persona's specific orders and bookings to know what kind of questions a "logged-in" user would ask. This allows you to create tasks where the user compares live market prices against their own previous mock-server purchases.

Historical Data: The "Logged-In" Context

Use the **Orders** section of the [Persona Visualizer](#) to craft trajectories that reference the user's past life events.

- **Comparison Tasks:** Find the current live price for the same brand of flowers the persona bought previously on the mock server.
- **Review & Feedback:** Have the persona look up a review or status for a movie they watched or a hotel they booked in their history.
- **Restocking Logic:** Ask for the current availability of an item that the persona previously purchased to see if it is still the cheapest option available.

Mapping Strategy

When you receive a persona like **Marcus Johnson**, map your trajectory in three layers:

1. **Identity Layer (Static Verification):** The model must correctly identify the user via tools like `get_current_user`. Any ID or location used by the agent must be grounded in the active persona's profile.

2. **Interest Layer (Semantic):** Use the Persona's interests to guide the intent. Since Marcus is a heavy gamer and rock climber, he wouldn't just ask for "shoes"; he would ask for "technical specifications and weight of climbing shoes" or "frame rates for the latest RPG release."
3. **Constraint Layer (Contextual):** Use the "Orders" section of the Visualizer to create contextually grounded information requests. Marcus has a history of gaming and climbing purchases. A high-quality prompt might say: "I see in my history that I bought a climbing harness last month; find me a review of the gym I visited in that same order, and then find the cheapest chalk bag that matches my existing gear."

Important Reminders

- **Be the User:** The agent directly accesses the active persona's data. Pretend you are logged in as that person. Review previous purchases, orders, and bookings in the Visualizer to know what questions the persona would realistically ask (e.g., "Is the price of the GPU I bought last year still the same?").
- **No Copy-Paste Biographies:** Do not simply paste the Persona Description into the prompt. Translate those traits into a natural request.
- **Cross-Server Utility:** Use mock data to establish what the user "has" and live data to find what is "new" or "current".
- **Diversity is Key:** Every Marcus Johnson task should be different. One day, he's checking programming books; the next, he's looking up the price of flowers he bought previously.

Persona Integration Checklist

Before submitting your task, run through these five questions to ensure your Persona is handled correctly:

1. Did I avoid having the persona introduce themselves? Does the prompt sound like someone who is already logged into their own account?
2. Does the request match the Persona's lifestyle while staying within the assigned Domain and Lens (Enterprise vs. Consumer)?
3. Is the information I'm seeking related to the persona's **Specialty Stores**? If they are a climber, are they checking a sports store? If they are a gamer, are they checking tech specs?
4. Did I avoid "copy-pasting" the persona description?
5. Does the "Why" of the request match the persona's biography and interests? (e.g., A "Tech Enthusiast" asking for advanced technical specs rather than just "a good deal.")

Persona-Driven Prompting: Good vs. Bad Examples

Using **Marcus Johnson** (Software Engineer, Gamer, Rock Climber) as our subject:

Feature	❌ The "Generic" Fail	✅ The "Persona-Expert" Pass
User Prompt	"I am Marcus Johnson. Can you see if there are any cheap climbing shoes in stock?"	"I'm planning a trip to Castle Rock this weekend. Check the stock for Scarpa Instinct VS climbing shoes and compare the price to my last order."
Logic	Too generic; ignores Marcus's specific hobby details and his purchase history.	Uses Lifestyle Data (Rock Climbing) and History Data (previous orders) to create a complex, high-utility request.
Grounding	Asks for personal info that the agent already has access to.	Acts as the logged-in user, referencing "my last order" without redundant self-introduction.
Tool Use	Calls a generic search tool without context.	Calls specific Specialty Stores (like <code>sporting_goods</code>) and uses <code>get_order_history</code> to retrieve baseline price data.

How to Avoid Common "Persona" Errors

- **Don't Over-Prescribe the Persona:** Focus on the **Outcome** (e.g., did they find the correct tech gear?) rather than the "subjective" tone.
- **Atomic History Checks:** If the prompt asks to compare a current price to a previous purchase, create a specific **Multi-Step Planning** rubric to verify that the model correctly identified the price from the order history tool output.